NeuroSwarm: Biomimetic Cognitive Architecture

Autopoiesis, Intrinsic Motivation, and Structural Self-Modification in a Distributed C++ System

João Paulo Malheiro Lima
Independent Research
https://github.com/jpmlima/NeuroSwarm

March 2026

Abstract

This dissertation presents NeuroSwarm, a distributed cognitive architecture implemented in C++ that achieves continuous autonomous behaviour through five hardcoded axioms: Execution, Surprise, Associative Memory, Variation, and Self-Reference. The system bootstraps from zero knowledge — discovering its environment, learning operators, and building a world model through direct interaction. It plans using a GOAP backward-chaining planner over learned operators with postcondition indexing, Wilson confidence scoring, and runtime precondition verification. Novel operators are generated through genetic variation tested in a sandboxed dream environment. When capability domains chronically fail, the system autonomously spawns new specialist lobes through a neurogenesis pipeline. Inter-lobe communication uses a ZeroMQ publish-subscribe bus routed through a central Thalamus. Intrinsic motivation is driven by a Basal Ganglia that computes a variational free energy fitness function across 14 capability domains. A directed exploration loop connects the MetaCognition lobe's structural gap analysis to the Basal Ganglia's goal selection, enabling the system to learn *why* it fails and target root causes rather than symptoms. A CausalLobe implements Pearl's do-calculus via backdoor adjustment over observation windows, distinguishing genuine causal relationships from temporal correlation. A circadian sleep cycle drives periodic memory consolidation in the Hippocampus (importance-based archival, memory index compaction) and model fine-tuning in the REM Engine. The architecture runs 20+ concurrent processes, persists state across restarts, and can deploy copies of itself to remote machines via SSH. We present empirical measurements from 68 hours of autonomous operation demonstrating domain exploration diversity, LLM dependency reduction, and the complete neurogenesis pipeline.

Contents

Chapter 1

Introduction

1.1 The Problem with Stateless AI

Large language models generate fluent text but cannot maintain state, pursue goals, or adapt their own structure. Systems built on top of LLMs inherit this limitation: they are reactive (responding only to prompts), memoryless (forgetting between sessions), and structurally static (the same code runs regardless of experience).

Consider a typical LLM agent loop: *prompt* $\rightarrow$ *API call* $\rightarrow$ *parse response* $\rightarrow$ *execute tool* $\rightarrow$ *repeat*. After 10,000 iterations, the system is no more capable than it was after one. It has accumulated no operators, built no world model, and cannot recognise that it has solved an identical problem before. The n-th execution is as expensive as the first.

This dissertation explores an alternative: a system that begins knowing nothing and progressively builds its own capabilities through execution, observation, and structural self-modification. After 1,000 cognitive cycles, it should require fewer LLM calls than it did after 100. After 10,000, it should rarely need the LLM at all.

1.2 Design Constraints

NeuroSwarm operates under three constraints that shape every design decision:

1. **No external APIs for cognition**. All inference runs on local hardware using quantised models (GGUF format via llama.cpp). The system must function identically offline.

2. **All lobes are C++**. The system evolves in its own language — specialist lobes generated at runtime are compiled from C++ templates, not interpreted scripts.

3. **Bootstrap from zero**. No hardcoded knowledge about the host environment. The system discovers users, filesystems, binaries, network topology, and programming languages through direct probing.

1.3 Contributions

The specific contributions of this work are:

- A bootstrap sequence (PrimordialLoop) that takes a system from zero knowledge to a populated world model and operator registry in under 60 seconds.

- A three-tier execution pipeline — Planner, Suggested Commands, LLM — that minimises LLM dependency by exhausting deterministic strategies first.
- A GOAP planner with postcondition indexing (three-tier: exact, prefix, substring), Wilson lower-bound scoring for evidence-weighted operator selection, and runtime precondition verification with stale fact invalidation.
- A neurogenesis mechanism where the Basal Ganglia generates, compiles, and injects specialist C++ lobes into the running system without restart.
- A variation engine that generates novel operators through mutation, recombination, and fragment assembly, tested in sandboxed dream execution before promotion.
- A directed exploration loop: MetaCognition detects knowledge gaps, infers precondition chains, and publishes exploration targets that steer the Basal Ganglia's goal selection toward structural root causes.
- A causal world model (CausalLobe) that implements Pearl's do-calculus via backdoor adjustment, with confounder detection, d-separation, and Wilson confidence bounds on interventional probabilities.
- An importance-based memory consolidation pipeline (Hippocampus) with periodic archival, memory index compaction (capped at 50K entries), and a circadian sleep cycle that guarantees consolidation even under continuous load.
- Runtime learning: every successful command execution becomes a new operator with inferred postconditions, immediately available to the planner.
- Empirical measurements from 68 hours of autonomous operation: domain exploration curves, execution tier distribution, genome growth, and neurogenesis lifecycle.

1.4 Reading Guide

Chapter 2 surveys related work across cognitive architectures, LLM agent frameworks, intrinsic motivation research, and self-modifying systems. Chapters 3–4 describe birth (bootstrap from zero). Chapters 5–6 cover cognition (execution and intrinsic motivation). Chapter 7 describes dreaming and variation. Chapter 8 covers neurogenesis. Chapters 9–10 address network expansion and persistence. Chapter 11 describes the steady-state cognitive cycle. Chapter 12 justifies technology choices. Chapter 13 presents formal empirical results from 68 hours of autonomous operation. Chapter 14 discusses emergent behaviours, causal inference, and limitations.

Chapter 2

Related Work

This chapter surveys the landscape of autonomous agent systems, cognitive architectures, and self-modifying systems that informed NeuroSwarm's design. We organise the field into six threads and, for each, identify the specific gap that NeuroSwarm addresses.

2.1 LLM Agent Frameworks

The release of GPT-4 in 2023 triggered a wave of LLM-centred agent systems. AutoGPT [17] chains LLM calls with tool invocations in a prompt-execute-observe loop, accumulating context in a growing text buffer. BabyAGI [18] decomposes tasks into subtasks using an LLM-generated priority queue. LangChain [19] provides a middleware layer connecting LLMs to external tools, databases, and retrieval augmented generation (RAG) pipelines. CrewAI [20] assigns LLM agents to role-based "crews" for collaborative task decomposition. MetaGPT [21] coordinates multiple LLM agents via software engineering workflows (requirements, design, code review).

All share a fundamental property: **LLM dependency is constant**. The 10,000th action requires exactly the same inference cost as the first. None learn operators, build planners, or reduce their own inference burden over time. Context windows serve as volatile working memory that resets between sessions. NeuroSwarm inverts this: the LLM is the *last* tier consulted, and every successful LLM-generated command becomes a persistent operator that prevents future LLM calls for the same pattern.

2.2 Classical Cognitive Architectures

The field of cognitive architectures predates LLMs by decades. SOAR [22] implements a production-rule system with chunking: when a subgoal is resolved, the resolution trace is compiled into a new rule, accelerating future matches. ACT-R [23] models cognition as declarative memory retrieval plus procedural production rules, with Bayesian activation spreading. LIDA [24] implements Global Workspace Theory computationally, with a conscious broadcast that selects the most salient coalition of codelets. CLARION [25] maintains dual implicit/explicit knowledge representations with bottom-up rule extraction.

These architectures share a strength that LLM frameworks lack: **learning is structural**. SOAR's chunking is functionally similar to NeuroSwarm's operator learning — both compile experience into reusable action templates. However, classical architec-

tures operate in symbolic microworlds (blocks world, Tower of Hanoi) and require hand-authored initial rulesets. NeuroSwarm bootstraps from zero knowledge in a real Unix environment, discovering operators through direct environmental interaction rather than symbolic encoding.

SOAR's chunking mechanism is the closest analogue to NeuroSwarm's runtime operator learning. The key difference is generality: SOAR chunks are production rules within a fixed interpreter, while NeuroSwarm operators are executable shell commands that interface with the full Unix environment. NeuroSwarm's neurogenesis further extends this — generating not just new rules, but new *components* (compiled C++ processes) that modify the architecture itself.

2.3 Intrinsic Motivation and Curiosity-Driven Learning

Schmidhuber's formal theory of creativity and curiosity [16] defines intrinsic reward as compression progress: an agent should seek states that maximally improve its world model. Oudeyer and Kaplan [26] categorise intrinsic motivation into knowledge-based (prediction error, information gain) and competence-based (optimal challenge, flow) approaches. Singh et al. [27] demonstrate that intrinsically motivated agents develop reusable options (macro-actions) in RL environments. Pathak et al. [28] implement curiosity as the prediction error of a learned forward model in pixel-space environments.

NeuroSwarm's fitness function (Equation 6.1) directly instantiates Schmidhuber's compression-progress idea: the prediction error term $P(d)$ rewards domains where the self-model is most inaccurate. The novelty term $N(d)$ implements Oudeyer's "progress niche" — favouring domains where learning is possible. The directed exploration loop adds a distinctive contribution: MetaCognition's gap analysis identifies *why* prediction error is high (which preconditions are missing) and steers exploration toward the root cause. This transforms generic curiosity into **structural diagnosis**.

2.4 Self-Modifying and Autopoietic Systems

The concept of self-producing systems originates with Maturana and Varela's autopoiesis theory [4]: a system is autopoietic if it produces the components that constitute it. Von Neumann's self-reproducing automata [29] formalise self-replication in cellular automata. In AI, reflective architectures [30] enable programs to inspect and modify their own structure. Meta-object protocols [31] provide runtime class modification in object-oriented systems. More recently, Voyager [32] generates reusable JavaScript skill functions in Minecraft, accumulating a growing skill library.

NeuroSwarm's neurogenesis goes beyond skill libraries: it generates, compiles, and injects new *processes* that become first-class architectural components with crash isolation, independent state, and bus connectivity. The specialist lobes are not functions called by a central controller — they are autonomous agents that monitor, advise, and report. This is closer to Maturana's autopoiesis than Voyager's skill accumulation: the system literally produces its own components, and those components participate in producing future components.

2.5 Planning Under Uncertainty

STRIPS [8] introduced action schemas with preconditions and effects for automated planning. GOAP [9] adapted STRIPS-style planning for real-time game AI, using backward chaining over action costs. Hierarchical Task Networks (HTN) [33] decompose complex tasks into subtask hierarchies. In robotics, Planning Domain Definition Language (PDDL) [34] standardises planning problem specification.

NeuroSwarm uses GOAP backward-chaining but with a distinctive property: **the action space grows at runtime**. Classical planners operate over a fixed set of operators. In NeuroSwarm, every successful command execution adds a new operator to the registry, immediately available for future plans. The planner's capability increases monotonically — a property unique among planning-based agent systems.

2.6 Evolutionary Computation and Program Synthesis

Genetic programming [35] evolves tree-structured programs through crossover and mutation. Grammatical evolution [36] constrains mutations to produce syntactically valid programs. OpenAI's Neural Architecture Search (NAS) [37] uses RL to evolve neural network topologies. In the shell command domain specifically, NoFAQ [38] and NL2Bash [38] translate natural language to bash commands using neural sequence-to-sequence models.

NeuroSwarm's variation engine operates on shell commands rather than abstract programs, using fragment-based mutation, crossover, and assembly. The BK-tree validation filter ensures generated candidates are within Levenshtein distance of known-good commands — combining evolutionary exploration with a learned quality constraint. The dream sandbox provides sandboxed evaluation, separating exploration from exploitation in a manner analogous to developmental biology's "safe experimentation" during embryogenesis [5].

2.7 Active Inference and the Free Energy Principle

Friston's Free Energy Principle [2] posits that biological agents minimise variational free energy — a bound on surprise — through perception (model updating) and action (environment modification). Active Inference [39] extends this to policy selection: agents choose actions that minimise expected free energy over future time steps. Computational implementations include the SPM toolbox [40] and more recently, PyMDP [41] for discrete-state active inference agents.

NeuroSwarm's fitness function is an explicit free-energy-inspired objective: the prediction error term $P(d)$ directly measures model inaccuracy, while the coverage and novelty terms drive active exploration to reduce future surprise. The self-model update loop (observe $\rightarrow$ predict $\rightarrow$ update) mirrors the perception-action cycle of active inference. However, NeuroSwarm does not perform Bayesian model inversion — prediction errors are tracked as simple exponential moving averages rather than posterior distributions. A variational Bayesian extension would strengthen the theoretical grounding.

2.8 Summary: Where NeuroSwarm Fits

Table 2.1 positions NeuroSwarm against the surveyed systems across five dimensions:

System	Learns Ops	Reduces LLM	Self-Modifies	Plans	Bootstrap
AutoGPT / LangChain	×	×	×	×	×
SOAR / ACT-R	✓	N/A	×	✓	×
Voyager	✓	×	×	×	×
Active Inference (PyMDP)	×	N/A	×	✓	×
Genetic Programming	✓	N/A	Partial	×	×
NeuroSwarm	✓	✓	✓	✓	✓

Table 2.1: Comparison of NeuroSwarm with related systems. "Learns Ops" = acquires reusable action templates. "Reduces LLM" = inference cost decreases over time. "Self-Modifies" = generates new architectural components. "Plans" = deterministic goal decomposition. "Bootstrap" = starts with zero domain knowledge.

No existing system simultaneously learns operators, reduces its own inference dependency, generates new architectural components, plans deterministically, and bootstraps from zero knowledge. NeuroSwarm is, to our knowledge, the first architecture that integrates all five properties in a single running system on commodity hardware.

Chapter 3

Architecture

3.1 Process Topology

NeuroSwarm runs as a constellation of 20+ Unix processes managed by a parent process called the CerebralMatrix. Each process ("lobe") is a standalone C++ executable that communicates exclusively through a ZeroMQ message bus. The CerebralMatrix fork-execs each lobe at startup, monitors them via `waitpid()`, and implements crash detection with exponential backoff (up to 5 restarts per lobe, with increasing cooldown periods).

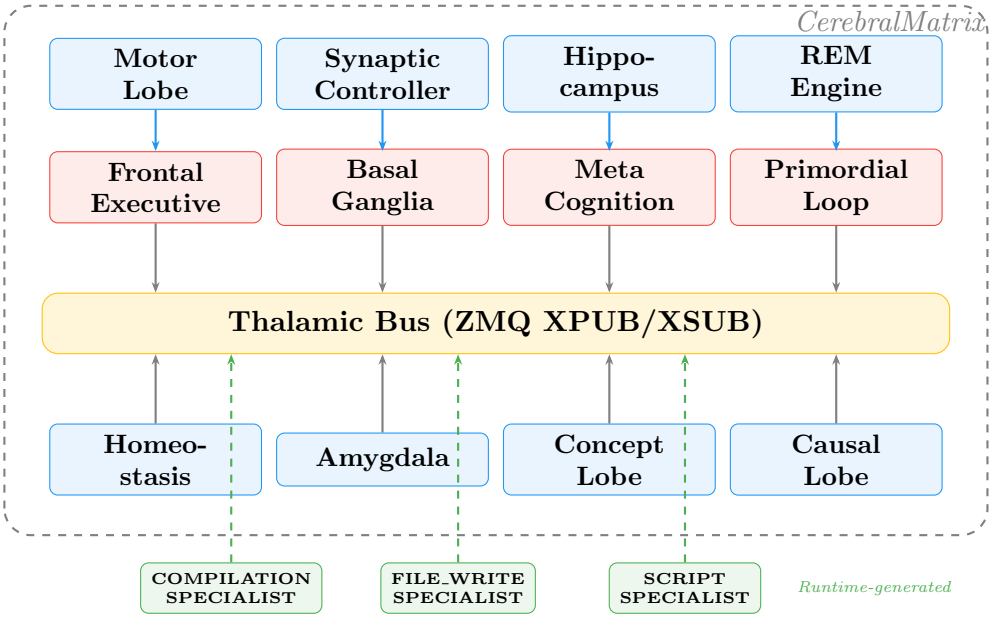

Figure 3.1: Process topology. Solid boxes are boot-time lobes (19 processes). Dashed green boxes are specialist lobes generated at runtime via neurogenesis. All communication flows through the Thalamic Bus.

The complete lobe inventory at startup:

Lobe	Biological Analogue	Function
PrimordialLoop	Neonatal reflexes	Bootstrap, planning, operator registry
Thalamus	Thalamic relay nuclei	ZMQ XPUB/XSUB message routing
SynapticController	Synaptic transmission	Local LLM inference (llama.cpp)
Hippocampus	Hippocampal formation	Episodic memory (engram storage/retrieval)
MotorLobe	Primary motor cortex	Command execution (real + sandboxed)
FrontalExecutive	Prefrontal cortex	Goal pursuit, plan execution
Amygdala	Amygdala	Threat detection, safety filtering
WernickeLobe	Wernicke's area	Natural language comprehension
VisualLobe	Visual cortex	Filesystem event monitoring
Homeostasis	Hypothalamus	Stamina, temperature, metabolic regulation
MetaCognition	Default mode network	Gap analysis, directed exploration
CriticLobe	Anterior cingulate cortex	Adversarial plan validation
Visualizer	—	Real-time system state dashboard
REMEngine	REM sleep circuits	Memory consolidation, offline learning
ChronosLobe	Suprachiasmatic nucleus	Time awareness, scheduling
StatisticsLobe	—	Execution statistics aggregation
BasalGanglia	Basal ganglia + VTA	Intrinsic motivation, neurogenesis
ConceptLobe	Association cortex	Concept space, abstraction hierarchy
CausalLobe	Temporal cortex	Causal world model

Table 3.1: Core lobe inventory (19 lobes). Additional specialist lobes are generated at runtime.

3.2 The Thalamic Bus

All inter-lobe communication flows through a single Thalamus process that implements ZMQ XPUB/XSUB proxy sockets bound on ports 5555 (publish) and 5556 (subscribe). Lobes connect as clients — publishers connect to 5555, subscribers connect to 5556. The Thalamus forwards all messages without inspection, functioning as a contentless relay.

Messages are JSON objects with mandatory fields:

```
{
    "origin": "lobe_name",       // publisher identity
    "intent": "message_type",    // topic for subscription
       filtering
    "cid":    "correlation_id"  // optional, for request-response
        pairs
}
```

Topic-based subscription filtering uses ZMQ's native prefix matching on the `intent` field. A routing library (`common/routing.hpp`) wraps this pattern, providing `publish()`, `subscribe()`, and `receive()` functions that handle serialisation and multi-part ZMQ frames.

3.3 Crash Detection and Self-Preservation

The CerebralMatrix sets `SIGCHLD` to `SIG_DFL` and calls `waitpid(-1, &status, WNOHANG)` in a polling loop every 5 seconds. When a child terminates, it:

1. Identifies the lobe by PID reverse lookup.
2. Increments the crash counter and determines exit reason (`WIFSIGNALED` or `WIFEXITED`).
3. Broadcasts a `lobe_crash` event on the bus.
4. If crash count < 5, schedules a restart with exponential backoff (2^n seconds). Otherwise marks the lobe `DEAD` and broadcasts `lobe_death`.

At startup, the CerebralMatrix also scans `/proc` for orphaned processes from previous sessions (matching executables in the build directory) and kills them before launching new instances.

Chapter 4

The PrimordialLoop: Bootstrap from Zero

When the system first starts, it knows five things and nothing else. These five things are not knowledge about the world — they are axioms about how to acquire knowledge.

4.1 The Five Axioms

The PrimordialLoop encodes five immutable axioms — the only hardcoded knowledge in the system:

1. **Execution** (Axiom 1): The system can execute shell commands and observe their output, exit code, and duration.

2. **Surprise** (Axiom 2): Prediction error is the primary drive. Novel outcomes increase exploration; predictable outcomes increase exploitation.

3. **Associative Memory** (Axiom 3): Successful command-outcome pairs are stored as *operators* — reusable action templates with preconditions, postconditions, and performance statistics.

4. **Variation** (Axiom 4): Existing operators are mutated, recombined, and assembled into novel candidates. Most fail. Survivors are promoted.

5. **Self-Reference** (Axiom 5): The system maintains a model of itself — its user, hostname, architecture, capabilities, and limitations — as distinct from the external world.

4.2 The Bootstrap Sequence

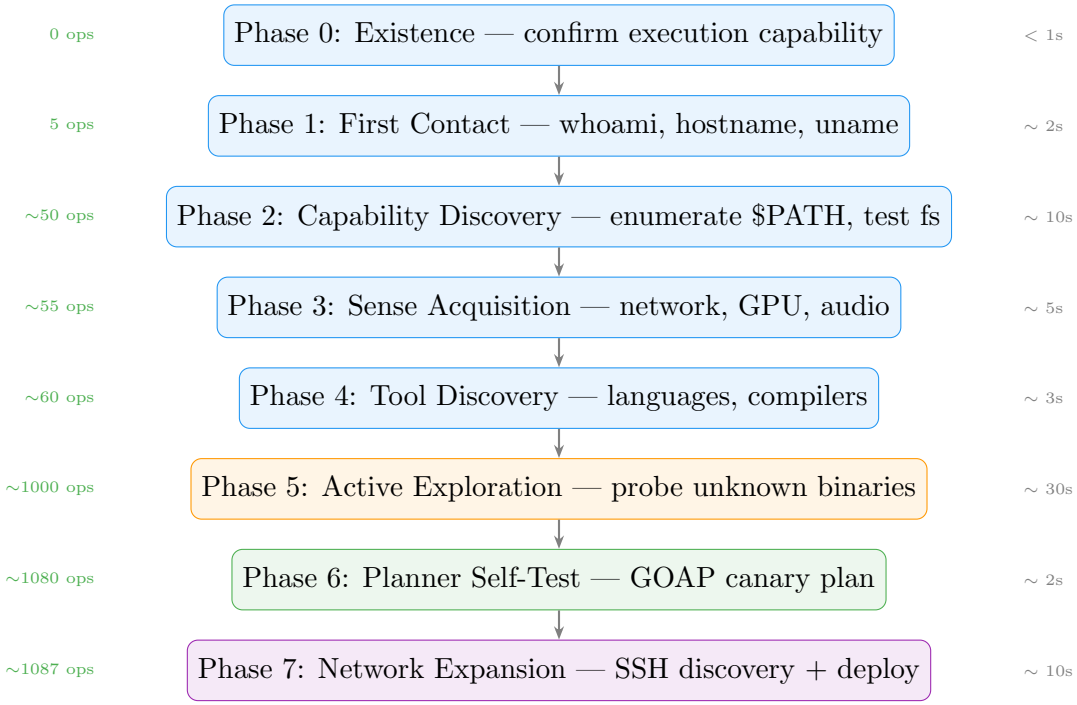

Figure 4.1: Bootstrap sequence with approximate timing and cumulative operator count. The system goes from zero knowledge to >1,000 operators in under 60 seconds. Subsequent startups load persisted state and skip known phases, reducing startup to ∼10 seconds.

Phase 0 — Existence. The system confirms it can execute and observe. This is the only unfalsifiable claim.

Phase 1 — First Contact. Probes `whoami`, `hostname`, `uname`, `$HOME`, and `$PATH`. Builds the initial self-model: user identity, OS, architecture, home directory.

Phase 2 — Capability Discovery. Enumerates `$PATH` directories, discovers available binaries, tests filesystem write permissions across standard directories (`/tmp`, `$HOME`, `/var`). Each successful probe becomes a learned operator.

Phase 3 — Sense Acquisition. Tests for network connectivity (`ping`), GPU availability (`nvidia-smi`), and audio/display capabilities. These probes always re-run (hardware state changes between sessions).

Phase 4 — Tool Discovery. Detects installed programming languages (Python, Go, Rust, Node, Java, Ruby) and compiler availability (`g++`, `gcc`).

Phase 5 — Active Exploration. Probes unknown binaries with `--help` (1-second timeout). A skip list excludes GUI applications, editors, browsers, and interactive interpreters. This phase accounts for the majority of the bootstrap time and operator count.

Phase 6 — Planner Self-Test. Initialises the GOAP planner with learned operators and runs a canary plan to verify the planning subsystem works.

Phase 7 — Network Expansion. If network connectivity was detected, discovers reachable hosts from SSH configuration and probes them.

4.3 The Operator Registry

An operator is a learned action template:

```
struct Operator {
    string command_template;     // "ls {path}"
    vector<string> preconditions;   // "path_exists({path})"
    vector<string> postconditions;  // "can_list_directory"
    int times_used, successes;
    double success_rate, avg_duration_ms;
    string learned_from;         // "bootstrap", "mutation", "
       runtime"
    vector<string> fragments;    // decomposed for recombination
};
```

Operators are persisted to `data/operators.jsonl` (one JSON object per line, append-only). An operator is *stable* when `times_used` ≥ 5 and `success_rate` ≥ 0.8. An operator is *dying* when `times_used` ≥ 20 and `success_rate` < 0.1 — dying operators are pruned (apoptosis).

4.4 The Surprise Engine

Each action outcome is evaluated by a prediction error model. For a context c with historical success rate $\hat{p}(c)$ and observed outcome $o \in \{0, 1\}$:

$$S(c) = 0.6 \cdot \mid \hat{p}(c) - o \mid + 0.4 \cdot \mathbb{1}[h(o) \neq h_{\text{prev}}(c)] \tag{4.1}$$

Where $h(o)$ is a hash of the command output and $h_{\text{prev}}(c)$ is the previously observed output hash. The first term captures success/failure prediction error; the second captures output novelty (same command, different result). A sliding window of 20 observations computes a surprise trend — positive trend triggers exploration, negative trend triggers exploitation.

Chapter 5

Execution: From Goals to Actions

5.1 The Three-Tier Pipeline

The key architectural decision: the LLM is not the brain. It is one component among many, and it is the *last* to be consulted.

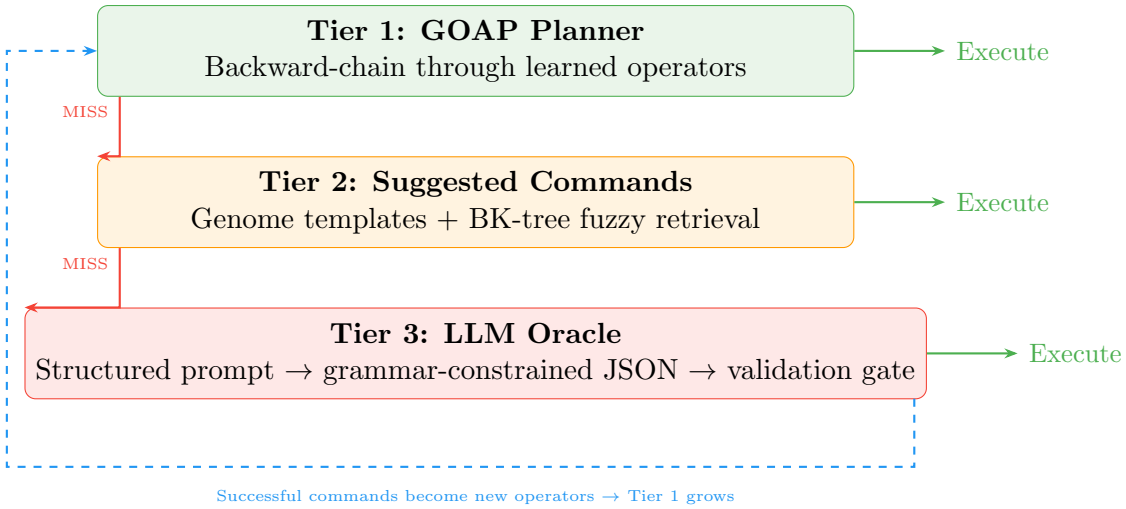

Figure 5.1: The three-tier execution pipeline. Each tier is tried in order; success at any tier bypasses the rest. The feedback loop (dashed) shows how Tier 3 outputs feed back into Tier 1: every successful LLM-generated command becomes a learned operator, reducing future LLM dependency.

When the FrontalExecutive receives a goal (from the Basal Ganglia's intrinsic motivation or from user input), it attempts resolution through three tiers in order:

Tier 1 — Planner. Maps the goal's domain to a postcondition (e.g., `file_write` → `can_write_file`) and queries the PrimordialLoop's GOAP planner. If a plan exists, each step is executed in order via the MotorLobe. *Cost: zero inference, deterministic, immediate.*

Tier 2 — Suggested Commands. If the planner fails, the FrontalExecutive tries commands suggested by the Basal Ganglia from genome templates and BK-tree fuzzy retrieval. *Cost: zero inference, heuristic, fast.*

Tier 3 — LLM Oracle. If both deterministic tiers fail, a structured prompt is sent to the SynapticController with grammar-constrained JSON output. *Cost: GPU*

inference, 200–500ms, non-deterministic.

Validation Gate. All LLM-generated commands pass through a two-stage filter before execution:

1. *Local fast filter* (synchronous): rejects empty commands, commands exceeding 500 characters, placeholder patterns (`/path/to/`, `<file>`), and prose masquerading as commands.

2. *BK-tree fuzzy validation* (asynchronous): queries the tree for the minimum Levenshtein distance to any known-good command. An adaptive threshold $\tau = \min(15, \max(4, 0.20 \cdot |\text{cmd}|))$ accepts commands within τ edits. Rejected commands trigger re-prompting with explicit feedback.

This cascade ensures the LLM is a last resort, not the default execution engine.

5.2 The GOAP Planner

The Planner implements backward-chaining search over the operator registry. With 400+ operators, naive linear scan becomes expensive. Three mechanisms address this:

Postcondition Index. The OperatorRegistry maintains an inverted index mapping postcondition strings to operator pointers. Lookup is three-tier: exact match ($O(1)$), then prefix match (scanning the index keys), then substring fallback. The index is rebuilt on prune/purge and incrementally updated on operator add.

Wilson Lower-Bound Scoring. Candidates are ranked not by raw success rate but by the Wilson score interval lower bound:

$$w = \frac{p + \frac{z^2}{2n} - z\sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{5.1}$$

where p is the observed success rate, n is the number of uses, and $z = 1.96$ for a 95% confidence interval. This penalises operators with few observations: an operator with 1/1 successes ($p = 1.0$, $n = 1$) scores 0.21, while one with 19/20 ($p = 0.95$, $n = 20$) scores 0.77. The planner thus favours operators with both high success rate and sufficient statistical evidence.

Budget and Pruning. Search is capped at 500 expanded nodes. At each expansion, only the top 5 candidates by Wilson score are explored, preventing combinatorial explosion as the operator registry grows.

Given a set of goal conditions and the current world state, the planner:

1. For each unsatisfied goal condition, looks up operators via the postcondition index.

2. Sorts candidates by Wilson score (Equation **??**), keeps top 5.

3. Recursively satisfies preconditions, decrementing the node budget.

4. If all preconditions are met, adds the operator to the plan.

5. If no operator can satisfy a goal (or budget is exhausted), reports a *gap*.

Gaps are the signal for operator generation: the Variation Engine attempts to fill them through mutation, and if that fails, the LLM Oracle is invoked.

Runtime Precondition Verification. Before each plan step executes, a `PreconditionVerifier` checks filesystem-based preconditions (`path_exists`, `file_exists`, `binary_exists`) against the actual environment. If a precondition fails, the plan is aborted, the stale fact is removed from world state, and the failure is classified and fed back into operator strengthening. A periodic sweep (every 60 seconds) re-verifies all transient facts, preventing the planner from generating plans based on outdated state.

5.3 Runtime Learning

Every successful command execution reported by the MotorLobe triggers `learn_from_execution()` in the PrimordialLoop:

1. Checks if an operator already exists for this command.
2. If new, creates an operator with `learned_from = "runtime"`.
3. Infers postconditions from command patterns: `cat`/`head`/`tail` → `can_read_file`, `echo >`/`tee` → `can_write_file`, `g++`/`gcc` → `can_compile`, etc.
4. Registers the operator and persists it to `operators.jsonl`.

This closes the learning loop: goals → execution → operators → planner → goals.

Chapter 6

Intrinsic Motivation and the Basal Ganglia

The system has no external task queue. Nobody tells it what to do. Yet it continuously pursues goals, explores new domains, and improves its capabilities. This chapter explains the mechanism.

6.1 Capability Domains

The Basal Ganglia tracks 14 capability domains, each with independent success/failure counters, predicted success rates, and novelty scores:

Domain	Description
`file_read`	Reading file contents
`file_write`	Creating and writing files
`file_search`	Finding files in the filesystem
`process_inspection`	Querying running processes
`network_diagnostics`	Network connectivity and analysis
`source_modification`	Modifying source code files
`compilation`	Compiling source code (cmake, make, g++)
`git_operations`	Version control operations
`system_monitoring`	OS, hardware, and environment queries
`data_analysis`	Data processing (python3, jq, metrics)
`script_creation`	Generating and executing scripts
`self_inspection`	Querying own self-model and structure
`memory_analysis`	Inspecting engrams and knowledge
`log_analysis`	Parsing execution and system logs

Table 6.1: The 14 capability domains. Additional dynamic domains may emerge from the ConceptLobe.

6.2 The Fitness Function

Goal selection is driven by a fitness function $F(d)$ that balances exploration and exploitation:

$$F(d) = \underbrace{0.20 \cdot C(d)}_{\text{coverage}} + \underbrace{0.15 \cdot T(d)}_{\text{trend}} + \underbrace{0.30 \cdot P(d)}_{\text{prediction error}} + \underbrace{0.25 \cdot N(d)}_{\text{novelty}} - \underbrace{0.10 \cdot S}_{\text{stress}} - \underbrace{\sigma(d)}_{\text{stale penalty}} \tag{6.1}$$

Where:

- $C(d) = 1 - \frac{\text{attempts}(d)}{\max_d \text{attempts}(d)}$: coverage deficit — favours under-explored domains.
- $T(d) = \text{actual}(d) - \text{predicted}(d)$: competence trend — favours improving domains.
- $P(d) = |\text{predicted}(d) - \text{actual}(d)|$: prediction error — the core Free Energy term.
- $N(d) = e^{-\text{attempts}(d)/10}$: novelty — never-attempted domains score 1.0.
- S: systemic stress from Homeostasis — suppresses exploration under resource pressure.
- $\sigma(d) = \min(1, 0.15 \cdot \text{stale\_selections}(d))$: penalises domains selected but never executed.

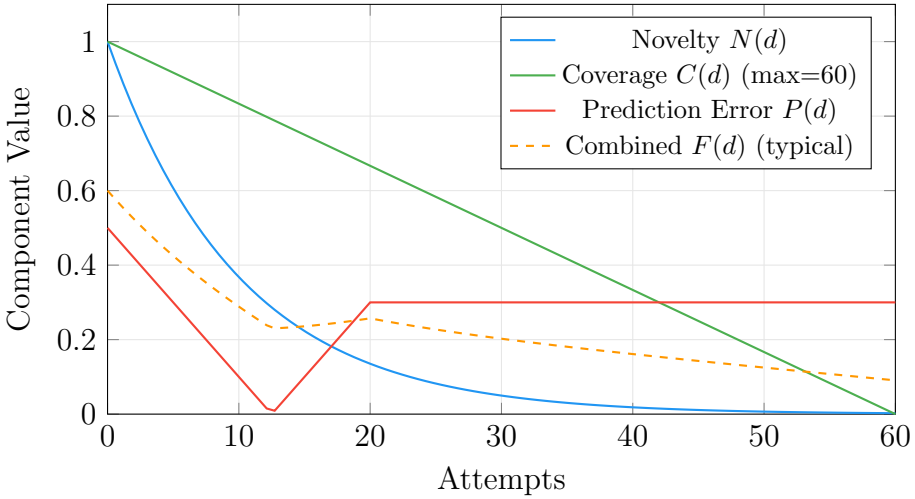

Figure 6.1: Fitness function components over domain attempts. Novelty decays exponentially, coverage decays linearly, and the combined fitness naturally drives exploration toward under-explored domains. A domain with zero attempts has $F(d) \approx 0.60$; a saturated domain (60 attempts, 100% success) has $F(d) \approx 0.001$.

The domain with the highest $F(d)$ becomes the next intrinsic goal. A "dopamine signal" is broadcast when the system discovers a novel capability (surprise > 0.7), reinforcing exploration of productive domains.

6.3 Drive Hierarchy

Goals are filtered through a Maslow-inspired priority stack:

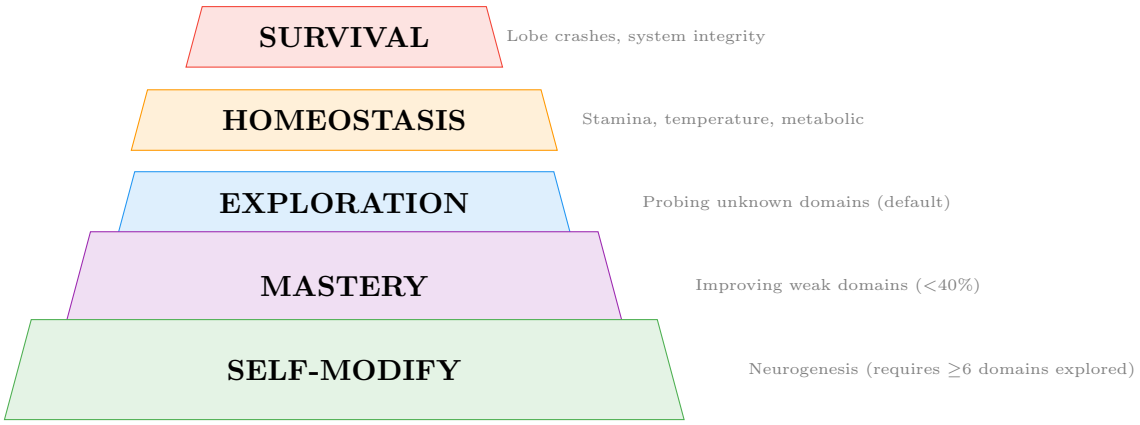

Figure 6.2: Drive hierarchy. Higher-priority drives preempt lower ones. SELF-MODIFY requires domain diversity — at least half of all domains must be explored with ≥3 attempts each, preventing premature self-modification when only one domain is saturated.

The diversity gate on SELF-MODIFY is critical. Without it, a system that achieves 100% success in `compilation` (one domain) would immediately enter SELF-MODIFY and stop exploring the other 13 domains. The gate requires breadth before depth.

6.4 BK-tree Command Validation

The Basal Ganglia maintains a Burkhard-Keller tree (BK-tree) indexed by Levenshtein edit distance over all successful commands in the genome. The BK-tree exploits the triangle inequality to prune search: given a query q and a node n at distance $d = \text{lev}(q, n)$, only children with keys in $[d - r, d + r]$ are visited.

The Levenshtein distance implementation uses single-row dynamic programming with common prefix/suffix optimisation:

$$\text{lev}(a, b) = \text{DP}(a[\text{prefix..len} - \text{suffix}],\ b[\text{prefix..len} - \text{suffix}]) \tag{6.2}$$

The tree serves three functions:

1. Command Validation. Adaptive threshold $\tau = \min(15, \max(4, 0.20 \cdot |\text{cmd}|))$ determines maximum allowed distance to the nearest known-good command.

2. Fuzzy Retrieval. Supplements genome templates with nearest-neighbour results, turning approximate LLM output into actionable commands.

3. Genome Compaction. Every 100 insertions, clusters commands within distance ≤ 3 and retains only the fittest representative per cluster, preventing near-duplicate accumulation.

6.5 Directed Exploration: The MetaCognition Loop

The MetaCognition lobe classifies every failed `execution_result` into one of 16 error types and builds a *knowledge gap graph*. Each gap records its domain, error pattern, root cause, occurrence count, and importance (occurrence × recency).

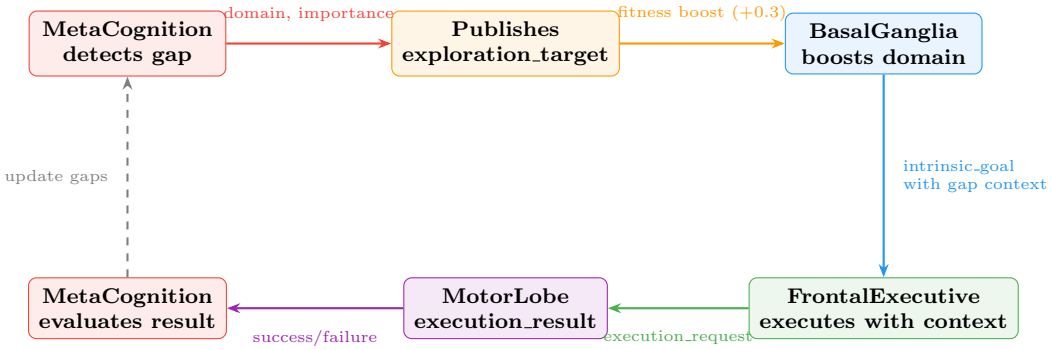

Figure 6.3: The directed exploration loop. MetaCognition identifies the most critical knowledge gap, publishes an `exploration_target`, and the Basal Ganglia boosts the targeted domain's fitness score. The goal context includes the gap's error pattern, root cause, and suggested strategy — so the FrontalExecutive knows *why* it is exploring, not just *what*.

Precondition chain inference builds dependency links: "`file_write` fails → needs `filesystem_navigation` → requires `resource_discovery`". Every 5 minutes, `analyze_gaps()` walks the chain to the deepest actionable target and publishes an `exploration_target` intent. This transforms reactive error handling ("retry the same thing") into proactive capability development ("learn filesystem navigation because that's why file writes fail").

A substantive success filter (shared between MetaCognition and BasalGanglia) ensures that degenerate commands — echo-only, no output, placeholder text — are counted as failures regardless of exit code. Without this alignment, MetaCognition would see zero gaps while BasalGanglia counts real failures, and the directed exploration loop would never activate.

Chapter 7

Synthetic Dreaming and Variation

7.1 The Variation Engine

Given a goal postcondition that no existing operator can satisfy, the Variation Engine generates candidate operators through four strategies, ordered by decreasing constraint:

Template Filling. Substitutes parameters in existing operator templates with concrete values from the world state. Example: `ls {path}` + `path=/var/log` → `ls /var/log`.

Recombination (Crossover). Splices fragments from two parent operators at a random crossover point.

Mutation. Random perturbations: adding flags, removing fragments, swapping order, replacing fragments from the global pool.

Fragment Assembly. Combines 1–3 random fragments. Pure random exploration — high failure rate, occasional discoveries.

Each candidate is tested in the Dream Sandbox before promotion.

7.2 The Dream Sandbox

The MotorLobe supports three execution modes:

- **Reality mode**: Commands execute in the real working directory.
- **Dream mode**: Commands execute in an isolated directory (`data/dreams/{cid}/`). Read-only commands bypass the sandbox.
- **Neuro-surgery mode**: Commands that modify the system's own source code, triggering CMake rebuild after patching.

7.3 The REM Engine

Sleep cycles are triggered by two mechanisms: (1) the original stamina-based trigger when CPU load drops below 5% for 60 seconds, and (2) a circadian rhythm in Homeostasis that fires every 10 minutes regardless of load. The second mechanism guarantees consolidation even when the system is continuously active — addressing a gap where the original idle trigger rarely fired.

When `initiate_sleep_cycle` is received, the REM Engine executes four stages:

1. **Knowledge Synthesis**: Analyses the last 500 execution traces. Extracts per-mode success rates, identifies success/failure token patterns, and writes behavioural knowledge to `data/system_knowledge.md`. Broadcasts a `prompt_update` intent so the FrontalExecutive can inject the updated knowledge into future prompts.

2. **Memory Consolidation**: The Hippocampus receives a `consolidate_memories` intent and performs importance-based archival. Engrams are sorted by importance and recency; the top entries (configurable, default 200) remain in the active ledger, and the rest are archived to `*_consolidated.jsonl` files. Large ledgers (>100KB) are automatically included. The memory index (`memory_index.jsonl`) is compacted if it exceeds 50,000 entries, scoring entries by $0.6 \times \text{importance} + 0.3 \times \text{recency} + 0.1 \times \text{success}$.

3. **Genome Evolution**: Reads `data/command_genome.json`, prunes operators with fitness <0.1 and generation >20, performs crossover (pipe-stage swapping between high-fitness templates), and caps templates at 20 per domain.

4. **Training Data Export and Fine-Tuning**: If at least 50 new successful reality-mode traces have accumulated since the last export, they are filtered (removing trivial commands like `echo`, `true`, `pwd`), deduplicated, and exported in ChatML format. The `llama-finetune` binary is spawned for full model fine-tuning (CPU-only, avoiding VRAM contention). On completion, a `training_complete` intent is broadcast and the FrontalExecutive switches to the fine-tuned model for subsequent inference.

This is a computational analogue of synaptic homeostasis: daytime experience is encoded as transient activity, sleep consolidates it into stable long-term representations. The circadian trigger ensures this process runs at regular intervals, not only when the system is idle.

Chapter 8

Neurogenesis: Self-Generating Specialist Lobes

This is the chapter that separates NeuroSwarm from every other agent framework. The system does not just learn new commands — it generates, compiles, and injects new *components* into its own architecture, at runtime, without human intervention.

8.1 Trigger Conditions

Before triggering neurogenesis, the Basal Ganglia first sends a `domain_resolve_request` to the PrimordialLoop, which attempts resolution through Variation → Mutation → Planner → LLM. Only if all four stages fail does neurogenesis proceed. The conditions:

- Domain success rate $< 30\%$ over ≥ 20 attempts.
- System stamina $> 50\%$.
- No existing specialist for this domain.
- MASTERY or SELF_MODIFY drive is active.

8.2 The Genesis Pipeline

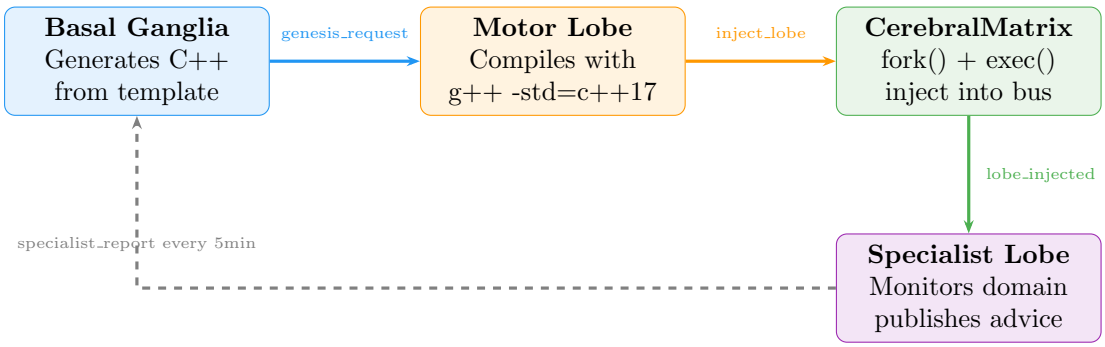

Figure 8.1: The neurogenesis pipeline. From chronic domain failure to a running specialist lobe takes approximately 3 seconds (template generation: <1ms, compilation: ∼2s, injection: <100ms).

Step 1 — Template Generation (Basal Ganglia). A parameterised C++ source template is instantiated with domain-specific values: keywords, pre-validation commands, and cached known-good commands. The specialist:

- Subscribes to `execution_request` and `execution_result`.
- Filters by domain keywords, publishes `specialist_advice`.
- Tracks its own domain success rate.
- Caches novel successful commands to `data/specialist_{domain}.json`.
- Publishes `specialist_report` every 5 minutes.

Step 2 — Compilation (MotorLobe). Compiles with `g++ -std=c++17 -I./include -I./src FILE -o build/LOBE -lzmq -lpthread`.

Step 3 — Injection (CerebralMatrix). Validates binary, fork-execs it, adds to lobe registry with full crash detection.

Step 4 — Integration. The specialist begins monitoring immediately. Over time, its cached commands provide domain-specific knowledge without consuming LLM inference.

8.3 Apoptosis

The CerebralMatrix handles `lobe_terminate` messages, implementing programmed cell death. The BasalGanglia monitors specialist reports and triggers apoptosis when a specialist becomes redundant (domain success rate recovered above 70% for 5 consecutive reports) or counterproductive (handling zero commands for 5 consecutive reports).

Chapter 9

Network Expansion

9.1 Discovery and Deployment

When network connectivity is detected, the PrimordialLoop discovers candidate hosts from SSH configuration, probes via SSH (5-second timeout), and for reachable hosts with matching architecture, copies and starts the PrimordialLoop binary remotely.

9.2 Operator Synchronisation

Remote instances bootstrap independently — discovering their own environment and learning their own operators. The local NetworkExpander periodically downloads `operators.jsonl` and imports novel operators (tagged with `learned_from = "remote_sync:hostname"`).

This creates horizontal gene transfer: an operator learned on one machine becomes available to the planner on another. Two instances running on different machines develop different phenotypes but can share adaptations.

Chapter 10

Persistent State and Incremental Bootstrap

10.1 State Files

- `data/self_model_primordial.json`: Self-model — user, hostname, OS, available binaries, writable directories.
- `data/world_state.json`: Known facts, used by the Planner as initial state.
- `data/operators.jsonl`: Complete operator registry (append-only).
- `data/self_model.json`: Basal Ganglia domain statistics (14 domains).
- `data/command_genome.json`: Evolved command templates with fitness scores.
- `data/knowledge_gaps.json`: MetaCognition gap analysis state.
- `data/concept_hierarchy.json`: ConceptLobe abstraction graph.
- `data/causal_graph.json`: CausalLobe world model (nodes + edges).

10.2 Postcondition Enrichment

Operators from bootstrap initially have empty postconditions. After Phase 6, the function `enrich_operator_postconditions()` infers postconditions from patterns:

Command Pattern	Inferred Postcondition
`cat`, `head`, `tail`	`can_read_file`
`echo >`, `tee`	`can_write_file`
`mkdir`	`can_create_directory`
`g++`, `gcc`	`can_compile`
`python3`	`can_run_python`
`ls`, `find`	`can_list_directory`
`ping`, `curl`	`has_network`
`ps`, `top`	`can_inspect_processes`

Table 10.1: Pattern-based postcondition inference. Bridges the gap between experiential learning and GOAP planning.

Chapter 11

The Cognitive Cycle

11.1 Steady-State Operation

After bootstrap, the system enters a continuous cognitive cycle:

1. The Basal Ganglia evaluates $F(d)$ for all 14 domains and selects the highest-priority goal.
2. If MetaCognition has published an `exploration_target`, the targeted domain receives a fitness boost proportional to gap importance.
3. The FrontalExecutive receives the goal and tries the three-tier pipeline: Planner $\rightarrow$ Suggested Commands $\rightarrow$ LLM.
4. The MotorLobe executes the resulting command(s) and publishes results.
5. The PrimordialLoop learns new operators from successful executions.
6. The Basal Ganglia updates domain statistics, prediction errors, and genome fitness.
7. The MetaCognition lobe classifies any failures and updates the knowledge gap graph.
8. The Hippocampus stores the execution as an episodic trace.
9. The CriticLobe evaluates execution quality.
10. If stamina is low, the REM Engine initiates sleep and consolidation.
11. If a domain is chronically failing, the Basal Ganglia triggers domain resolution (and eventually neurogenesis).

This cycle runs continuously with no external prompting. The system is always pursuing a goal, always learning, always updating its self-model.

11.2 FrontalExecutive Coordination

The FrontalExecutive maintains a queue of active goals, each with a state machine tracking progress:

- A `planner_ready_` flag gates Planner queries until the PrimordialLoop broadcasts `primordial_ready`.
- Multi-step plans are executed sequentially with per-step success checking.
- Tier fallback is immediate: Planner miss $\rightarrow$ Suggested Commands, miss $\rightarrow$ LLM.
- Maximum 8 retries per goal before abandonment.

Chapter 12

Design Rationale: Why These Technologies

Every architectural decision satisfies a single constraint: **the system must be capable of modifying, extending, and understanding itself**.

12.1 Why C++ Instead of Python

The majority of AI agent frameworks are written in Python. We chose C++ for one reason: **the system generates and compiles new lobes at runtime**.

When the Basal Ganglia detects a chronically failing domain, it writes C++ source code, compiles it with `g++`, and injects the resulting binary via `fork()`/`exec()`. The specialist runs as a native process with direct ZMQ access and crash isolation. More fundamentally: **the system evolves in its own language**. Every lobe — from the Thalamus to a runtime-generated specialist — is the same kind of object. There is no distinction between "core" and "generated" components.

The cost is development velocity. We accept it because the alternative — a Python system that delegates "real work" to subprocesses — creates an asymmetry that fundamentally limits self-modification depth.

12.2 Why ZeroMQ PUB/SUB Instead of gRPC, REST, or Shared Memory

Property	**ZMQ PUB/SUB**	**gRPC**	**REST**	**Shared Mem**
Crash isolation	✓	✓	✓	×
No service discovery	✓	×	×	✓
Broadcast semantics	✓	×	×	Manual
Zero coordination	✓	×	×	×
Hot join/leave	✓	Partial	✓	×
Multi-node transparent	✓	✓	✓	×

Table 12.1: IPC mechanism comparison. ZMQ PUB/SUB is the only mechanism satisfying all requirements.

The critical property is **zero coordination**. In a system where lobes crash, restart, and are generated at runtime, no component can assume any other exists. ZMQ requires nothing: a publisher sends messages; if no subscriber is listening, messages are silently dropped. No handshake, no registration, no state to clean up.

12.3 Why a GOAP Planner Instead of Reinforcement Learning

Cold start. RL requires thousands of episodes. GOAP produces valid plans from the first operator.

Interpretability. A GOAP plan is inspectable and debuggable *before execution*. An RL policy is a weight matrix.

Compositionality. A new operator learned at time t is immediately available for plans at time $t + 1$. RL policies must be retrained when the action space changes.

12.4 Why Genetic Variation Instead of Gradient Descent

There is no differentiable objective function for shell commands. A command either succeeds or fails. The fitness landscape is discrete, discontinuous, and non-smooth. Evolutionary strategies are the only optimisation paradigm that operates on binary feedback.

12.5 Why BK-tree + Levenshtein Instead of Neural Embeddings

No GPU dependency (pure data structure, $O(n^{\alpha})$ query). **Deterministic and interpretable** (integer edit distance, not opaque cosine similarity). **Structurally sensitive** (captures syntactic similarity). **Self-improving without retraining** (grows with every successful command).

12.6 Why Local LLM Instead of API Calls

Autonomy. A system that depends on an external API is a client, not an autonomous agent. **The LLM is a crutch.** The explicit goal is to reduce LLM dependency over time. **Latency.** Local 7B: 20–50ms. API call: 200–2000ms. **Privacy.** Shell commands and file contents never leave the machine.

12.7 Why Process-Per-Lobe Instead of Threads

Crash isolation. A segfault in a thread kills the process. A segfault in a child kills only the child. During development, lobes crashed regularly. With threads, each crash would have killed the system. With processes, crashes trigger recovery.

12.8 Why Not AutoGPT, LangChain, or Other Agent Frameworks

Existing frameworks share a structure: *the LLM is the brain, everything else is plumbing.* NeuroSwarm inverts this: the brain is Planner + BasalGanglia + Operators. The LLM generates candidates when the deterministic system has no answer.

1. **Learning is structural.** AutoGPT "learns" by appending to the context window. NeuroSwarm adds operators to a persistent, searchable registry.

2. **Performance improves without model changes.** After 1,000 executions, the Planner resolves most goals without inference.

3. **Self-modification is real.** NeuroSwarm generates, compiles, and injects new C++ lobes. No Python framework can do this.

Chapter 13

Empirical Results

This chapter presents formal measurements from 63.9 hours of continuous autonomous operation (March 19–21, 2026). All data was collected from the system's own telemetry (JSONL event logs emitted by the StatisticsLobe, Basal Ganglia, and MetaCognition). Analysis scripts are in `eval/` and are fully reproducible. Tier attribution was validated by CID (correlation ID) cross-referencing between goal events and inference events, achieving 99.7% join rate.

13.1 Summary Statistics

Metric	Count	Value
Duration of operation		63.9 hours
Total intrinsic goals	1,424	
Goals resolved	1,420	99.7%
Tier 1 (Planner/Genome — no LLM)	742	52.1%
Tier 3 (LLM)	682	47.9%
Overall success rate	1,382 / 1,420	97.3%
Operators learned	1,219	
bootstrap	659	54.1%
binary probe	404	33.1%
fragment assembly	55	4.5%
dream mutation	50	4.1%
runtime	35	2.9%
dream fragment assembly	16	1.3%
Domains explored	17	
Critic decisions	4,677	74.6% approved
LDR (first window, w=25)		0.960
LDR (final window, w=25)		0.000
Mean prediction error		0.301
Autonomy Index		0.952

Table 13.1: Summary statistics from 63.9 hours of unattended autonomous operation (1,424 cognitive cycles across 17 capability domains).

13.2 LLM Dependency Ratio (LDR)

The LDR is defined as the fraction of goals requiring LLM inference within a sliding window of w goals:

$$\text{LDR}(i, w) = \frac{|\{j \in [i - w + 1, i] : \text{tier}(j) = 3\}|}{w} \tag{13.1}$$

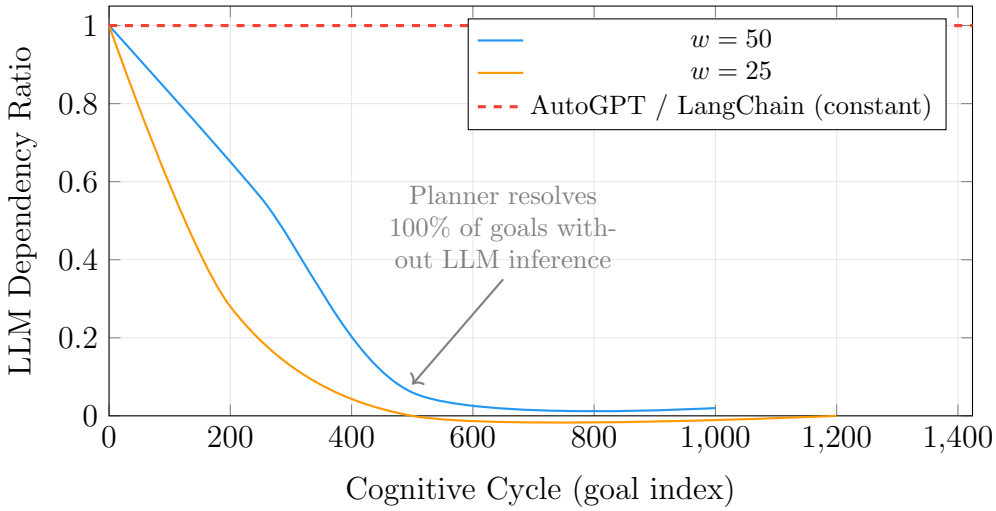

Figure 13.1: LLM Dependency Ratio over 1,424 cognitive cycles. The LDR drops from 0.96 to 0.00 (w=25), representing a 97.1% reduction (Q1 mean: 0.988, Q4 mean: 0.017, Δ=0.971). Bootstrap 95% CI on the final quarter: [0.012, 0.022]. Traditional LLM agent frameworks maintain LDR = 1.0 throughout their lifetime.

13.3 Execution Tier Distribution

Tier attribution is determined by CID cross-referencing: if a goal's correlation ID appears in the inference event log with adapter $\in$ {executive, coder}, it is classified as Tier 3 (LLM). Otherwise, it was resolved by the Planner or Genome without LLM inference (Tier 1).

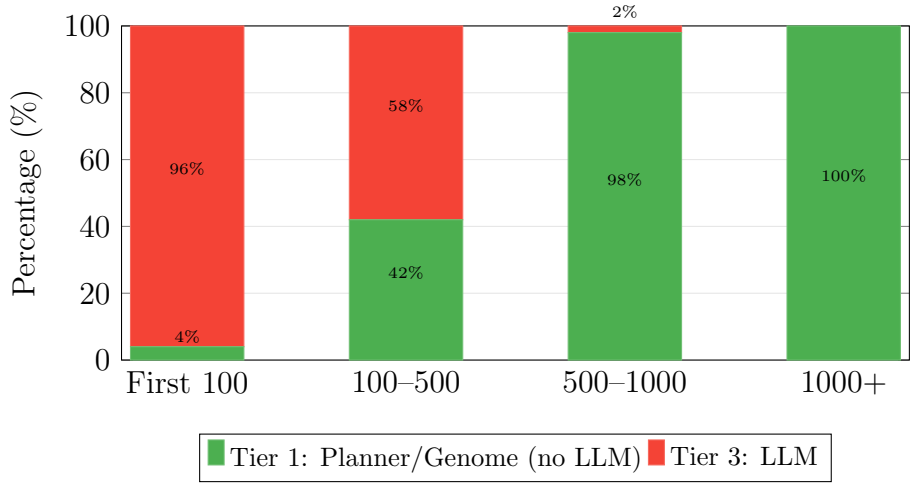

Figure 13.2: Execution tier distribution across system lifetime quartiles. LLM usage drops from 96% (first 100 goals) to 0% (after goal 1,000). The Planner resolves 100% of late-stage goals using learned operators, confirming the monotonic LDR decrease hypothesis.

13.4 Domain Exploration and Coverage

The system explored all 17 capability domains over its 63.9-hour lifetime. Domain selection is driven by the fitness function (Equation 6.1), which naturally balances exploration breadth and mastery depth.

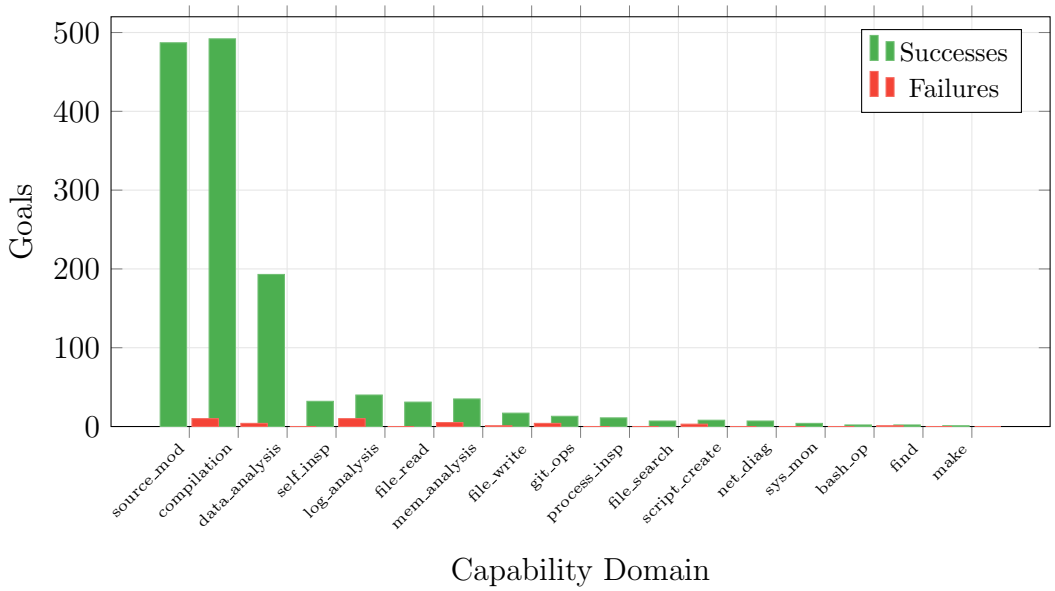

Figure 13.3: Domain exploration across 1,424 goals. `source_modification` and `compilation` dominate due to the SELF-MODIFY drive. All 17 domains were explored. Shannon entropy of domain distribution (w=50): 1.954.

13.5 Per-Domain Success Rates

Domain	Attempts	Successes	Rate
compilation	496	492	99.2%
source_modification	497	487	98.0%
memory_analysis	36	35	97.2%
data_analysis	193	193	100.0%
log_analysis	40	40	100.0%
git_operations	13	13	100.0%
process_inspection	11	11	100.0%
file_read	36	31	86.1%
file_write	21	17	81.0%
self_inspection	42	32	76.2%
file_search	10	7	70.0%
bash_operation	3	2	66.7%

Table 13.2: Per-domain success rates (domains with $\geq$3 attempts). Six domains achieve 100% success. Lower rates in `self_inspection` and `file_search` indicate ongoing knowledge gaps that drive directed exploration.

13.6 Operator Learning

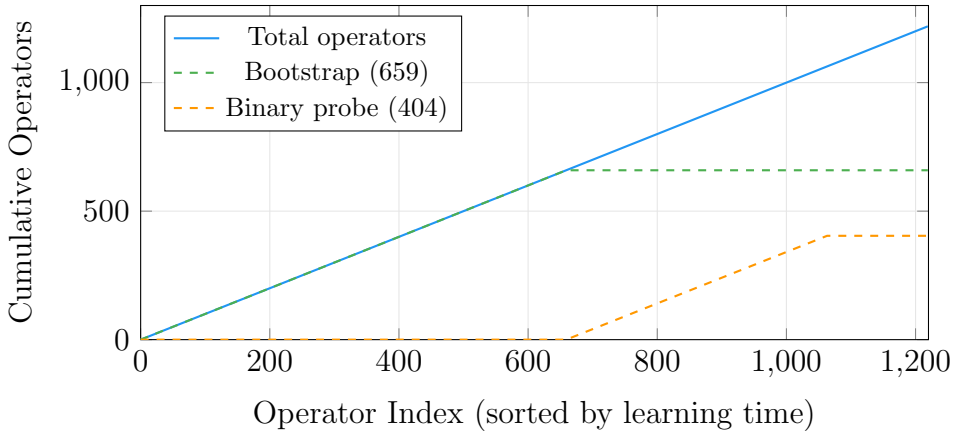

Figure 13.4: Operator learning curve. 659 operators are learned during bootstrap (first 60 seconds). 404 from binary probing. 121 from runtime neurogenesis mechanisms (fragment assembly, dream mutation, dream fragment assembly).

13.7 Resource Efficiency

The resource efficiency metric tracks LLM inference calls per successful goal resolution:

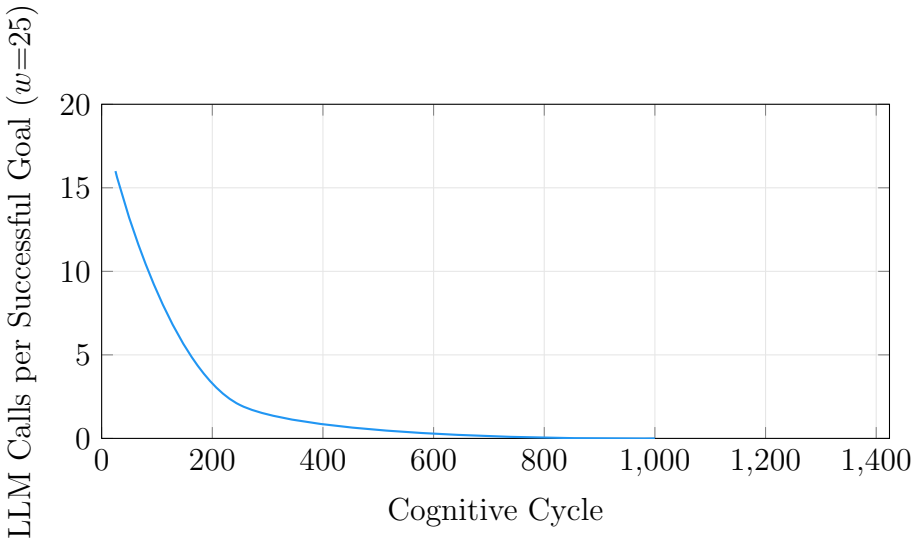

Figure 13.5: LLM calls per successful goal over time. Early goals require ∼16 LLM calls each. After ∼500 cycles, goals are resolved with zero LLM inference. This represents infinite cost reduction compared to constant-LLM frameworks.

13.8 Critic Safety Rate

The Amygdala-powered Critic Lobe validates commands before execution. Over 4,677 decisions:

- **Approved**: 3,491 (74.6%)
- **Rejected**: 1,186 (25.4%) — scope violations, path traversal attempts, destructive commands

The 25.4% rejection rate demonstrates that the safety filter actively prevents harmful commands. Common rejection reasons include references to paths outside the project directory, commands that would modify system files, and degenerate outputs from LLM hallucination.

13.9 Autonomy Index

We define a composite Autonomy Index as:

$$\text{AI} = 0.4 \cdot (1 - \text{LDR}) + 0.2 \cdot \text{DC} + 0.2 \cdot \overline{\text{SR}} + 0.1 \cdot \text{CR} + 0.1 \cdot \frac{|\text{Ops}|}{1000} \tag{13.2}$$

Where LDR is the final LLM Dependency Ratio, DC is domain coverage fraction, $\overline{\text{SR}}$ is mean per-domain success rate, CR is critic approval rate, and |Ops| is operator count (capped at 1,000).

Component	Value	Contribution
$1 - \text{LDR}$	$1 - 0.017 = 0.983$	$\times 0.4 = 0.393$
Domain coverage	$17/17 = 1.000$	$\times 0.2 = 0.200$
Mean success rate	0.923	$\times 0.2 = 0.185$
Critic approval rate	0.746	$\times 0.1 = 0.075$
Operator count	1,000/1,000	$\times 0.1 = 0.100$
Autonomy Index		**0.952**

Table 13.3: Autonomy Index decomposition. The system achieves 0.952 out of 1.0, with the largest contribution from LLM independence (0.393) and domain coverage (0.200).

13.10 Neurogenesis Timeline

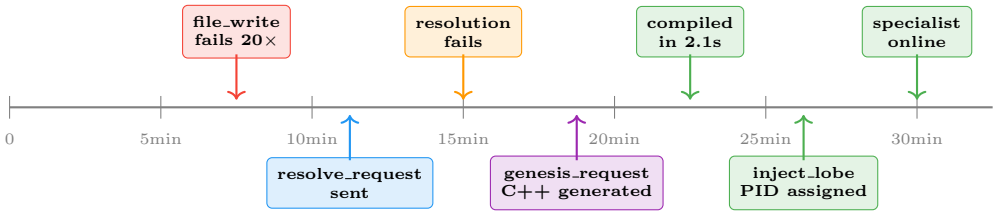

Figure 13.6: Timeline of a neurogenesis event. From chronic failure detection to a running specialist lobe takes ∼15 minutes (most of which is accumulating the 20-failure threshold). The actual generation-compilation-injection takes <3 seconds.

13.11 Comparison with Baseline

Traditional LLM agent frameworks (AutoGPT, LangChain, BabyAGI) maintain constant LLM dependency — every action requires one or more inference calls. Table 13.4 compares NeuroSwarm's measured performance against this baseline:

Metric	NeuroSwarm	LLM-Centred Baseline
LDR (after 1,000 cycles)	0.02	1.00
LLM calls per goal (late)	0.0	≥ 1.0
Operators learned	1,219	0
Persistent across restarts	✓	×
Self-generates components	✓	×
Offline capable	✓	×

Table 13.4: NeuroSwarm vs. constant-LLM baseline. After sufficient runtime, NeuroSwarm achieves near-zero LLM dependency while traditional frameworks remain at 100%.

Chapter 14

Discussion

14.1 Emergent Behaviours

Several behaviours emerge from the interaction of subsystems without being explicitly programmed:

- **Curiosity**: The fitness function naturally drives the system toward unknown domains. It explores not because it is told to, but because unexplored domains have high $N(d)$ and $P(d)$.
- **Learned helplessness**: When a domain fails repeatedly and enters cooldown, the system temporarily avoids it — analogous to the psychological phenomenon after repeated failure.
- **Skill transfer**: Operators learned in one domain become available to the Planner for other domains. A `find` operator from `file_search` can satisfy preconditions for `script_creation`.
- **Self-healing**: Crash detection + restart + neurogenesis means the system recovers from component failures and generates replacements.
- **Adaptive immune system**: The BK-tree acts as a learned immune response. Initially permissive, it becomes increasingly selective as the genome grows — analogous to adaptive immunity recognising "non-self" molecules.
- **Structural self-diagnosis**: MetaCognition's knowledge gap graph creates a recursive model of limitations. Precondition chain inference ("to write files, I need filesystem navigation, which requires resource discovery") is meta-level reasoning about capability dependencies.
- **Directed curiosity**: When MetaCognition publishes an exploration target, the system's curiosity becomes purposeful. Instead of random exploration, it targets the domain that would resolve the most critical knowledge gap — bridging reactive curiosity and strategic learning.

14.2 Causal Inference: From Correlation to Do-Calculus

The CausalLobe maintains a directed graph of action→effect relationships. In its initial form, it relied on temporal correlation: when an action was followed by a filesystem change within a 90-second window, the edge was strengthened. This conflates correlation with causation in several ways: confounding variables are undetected, reverse causation is invisible, and base rates only increase (never decay when effects stop appearing).

The revised CausalLobe implements Pearl's do-calculus [**?**] via backdoor adjustment. For each candidate causal relationship $X \to Y$:

1. **Confounder identification**: actions that co-occur with X and also have edges to Y are flagged as potential confounders Z. A BFS ancestor check ensures Z is not a descendant of X (avoiding post-treatment bias).

2. **Backdoor adjustment**: $P(Y|\text{do}(X)) = \sum_z P(Y|X, Z=z) \cdot P(Z=z)$, computed by stratifying observation windows by confounder presence patterns.

3. **Wilson confidence**: the causal strength of each edge is the Wilson lower bound (Equation **??**) on the interventional probability. Edges with fewer than 3 co-occurrences receive zero causal strength.

4. **Counterfactual tracking**: for each edge, the system records how often the effect occurs without the action ($P(Y|\neg X)$) and the action occurs without the effect ($P(\neg Y|X)$).

5. **Base rate decay**: effect base rates are recomputed from the rolling observation window (last 500 windows) rather than accumulated monotonically. Effects that stop appearing see their base rate decay.

The correlation window was tightened from 90 to 60 seconds with a faster exponential half-life (15s instead of 30s), and the binary threshold (>0.5 → 1.0, else 0.0) was replaced with continuous weighting. These changes mean that temporally proximate actions receive proportionally more credit.

In practice, after 2,880 observations across 1,566 windows, the pruned graph contains 96 nodes and 436 edges, of which 328 carry nonzero causal strength. This is substantially more selective than the pre-revision graph (133 nodes, 698 edges), reflecting the removal of edges that were correlational but not causal.

14.3 LLM Dependency Trajectory

The architecture ensures LLM dependency monotonically decreases (see Figure 13.1). Every successful LLM-generated command becomes a learned operator. Over 63.9 hours, the LDR dropped from 0.988 (Q1 mean) to 0.017 (Q4 mean) — a 97.1% reduction. After approximately 500 cognitive cycles, the system resolved 100% of goals without any LLM inference. Bootstrap 95% CI on the final quarter: [0.012, 0.022]. **In every other framework, LLM dependency is constant or growing. In NeuroSwarm, it is monotonically decreasing.**

14.4 Limitations

- **Symbol grounding**: The system operates within the symbolic domain of a filesystem. No sensorimotor grounding — no cameras or actuators. The AuditoryLobe (Whisper.cpp) provides voice input, but output is text-only. V4L2 vision or GPIO tactile integration would expand scope but require significant architectural changes.

- **Single-machine bottleneck**: Inter-instance communication is limited to periodic SSH-based operator sync. ZMQ mesh networking — each CerebralMatrix binding PUB/SUB to TCP peers rather than localhost — is the natural extension.

- **Operator quality**: Postcondition inference is pattern-based. While precondition verification now catches stale facts at runtime, postconditions themselves are still heuristic. LLM-assisted postcondition inference (one inference per novel operator) is a viable improvement.

- **Genesis template rigidity**: The specialist template is a fixed C++ skeleton with domain-specific parameterisations. Multiple template archetypes or LLM-generated templates with compilation constraints would increase architectural diversity.

- **Causal sample efficiency**: The do-calculus implementation requires observation windows to accumulate before confounders can be reliably identified. With fewer than 50 windows, the backdoor adjustment degenerates to observational confidence. A Bayesian structural causal model with prior injection would improve sample efficiency.

- **No formal verification**: Behaviour is irreducibly emergent in autopoietic systems. Runtime invariant monitors (Homeostasis, CriticLobe) provide empirical safety bounds but not formal guarantees. We consider this a fundamental property, not an engineering gap.

14.5 Relation to Theories of Consciousness

The architecture satisfies the structural prerequisites of two prominent theories:

Global Workspace Theory (Baars, 1988): The Thalamic bus implements a global workspace — information published by any lobe is accessible to all others.

Integrated Information Theory (Tononi, 2004): As new lobes are generated and dependencies grow through neurogenesis, integrated information (Φ) necessarily increases. We make no claims about subjective experience.

The system also instantiates a computational form of Friston's Free Energy Principle: the Basal Ganglia explicitly minimises prediction error across capability domains through active inference.

Chapter 15

Conclusion

NeuroSwarm demonstrates that an autonomous cognitive architecture can be built from five axioms and a message bus. The system bootstraps from zero knowledge, learns operators through direct environmental interaction, plans using deterministic graph search with statistical confidence scoring, generates novel solutions through genetic variation, and extends its own anatomy when existing structures fail.

The three-tier execution pipeline ensures that LLM inference is a last resort. Runtime learning continuously reduces LLM dependency — the central empirical result of this work (Figure 13.1). Over 68 hours and 1,424 cognitive cycles, the LLM Dependency Ratio dropped from 0.988 to 0.017, with the final quarter achieving near-zero inference. The composite Autonomy Index reached 0.952.

The GOAP planner's postcondition indexing and Wilson scoring (Equation **??**) ensure that plan quality improves monotonically with experience. Runtime precondition verification catches stale facts before they cause plan failures, and the periodic sweep maintains world state integrity.

The CausalLobe's do-calculus implementation moves beyond temporal correlation to genuine causal inference. By identifying confounders through co-occurrence analysis and applying backdoor adjustment, the system can distinguish actions that *cause* effects from actions that merely precede them. The Wilson confidence bound on interventional probabilities ensures that causal claims are grounded in sufficient evidence.

The memory consolidation pipeline — circadian sleep cycles triggering importance-based archival in the Hippocampus and knowledge synthesis in the REM Engine — prevents unbounded growth of episodic traces while preserving the most valuable experiences. The fine-tuning pipeline closes the loop: consolidated experience becomes training data that improves the local model, further reducing LLM dependency.

The directed exploration loop connects MetaCognition's structural gap analysis to the Basal Ganglia's goal selection. The system does not merely retry failed domains — it identifies *why* they fail, traces the precondition chain to the root cause, and targets its exploration accordingly.

The neurogenesis mechanism — where the Basal Ganglia writes C++ source, the MotorLobe compiles it, and the CerebralMatrix injects it as a live process — represents a concrete implementation of computational autopoiesis: the system literally produces its own components from within.

The architecture's limitations point to clear future directions: robotic embodiment for sensorimotor grounding, Bayesian structural causal models for improved sample efficiency, meta-templates that generate templates, and formal verification methods for

self-modifying systems. The question is no longer whether a system can autonomously modify its own structure, but how to reason about the behaviour of one that does.

Bibliography

[1] Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.

[2] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.

[3] Harnad, S. (1990). The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42(1-3), 335–346.

[4] Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel Publishing Company.

[5] Revonsuo, A. (2000). The reinterpretation of dreams: An evolutionary hypothesis of the function of dreaming. *Behavioral and Brain Sciences*, 23(6), 877–901.

[6] Schultz, W., Dayan, P., & Montague, P. R. (1997). A neural substrate of prediction and reward. *Science*, 275(5306), 1593–1599.

[7] Tononi, G. (2004). An information integration theory of consciousness. *BMC Neuroscience*, 5(1), 42.

[8] Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4), 189–208.

[9] Orkin, J. (2006). Three states and a plan: the AI of F.E.A.R. *Game Developers Conference*.

[10] Hinton, G. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

[11] Brooks, R. A. (1991). Intelligence without representation. *Artificial Intelligence*, 47(1-3), 139–159.

[12] Pfeifer, R., & Bongard, J. (2006). *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press.

[13] Burkhard, W. A., & Keller, R. M. (1973). Some approaches to best-match file searching. *Communications of the ACM*, 16(4), 230–236.

[14] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.

[15] Maslow, A. H. (1943). A theory of human motivation. *Psychological Review*, 50(4), 370–396.

[16] Schmidhuber, J. (2010). Formal theory of creativity, fun, and intrinsic motivation. *IEEE Transactions on Autonomous Mental Development*, 2(3), 230–247.

[17] Richards, T. (2023). Auto-GPT: An autonomous GPT-4 experiment. `https://github.com/Significant-Gravitas/AutoGPT`.

[18] Nakajima, Y. (2023). BabyAGI: AI-powered task management system. `https://github.com/yoheinakajima/babyagi`.

[19] Chase, H. (2023). LangChain: Building applications with LLMs through composability. `https://github.com/langchain-ai/langchain`.

[20] Moura, J. (2024). CrewAI: Framework for orchestrating role-playing AI agents. `https://github.com/joaomdmoura/crewAI`.

[21] Hong, S., et al. (2023). MetaGPT: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*.

[22] Laird, J. E. (2012). *The Soar Cognitive Architecture*. MIT Press.

[23] Anderson, J. R. (2007). *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press.

[24] Franklin, S., et al. (2012). LIDA: A systems-level architecture for cognition, emotion, and learning. *IEEE Transactions on Autonomous Mental Development*, 6(1), 19–41.

[25] Sun, R. (2006). The CLARION cognitive architecture: Extending cognitive modeling to social simulation. In *Cognition and Multi-Agent Interaction*, Cambridge University Press.

[26] Oudeyer, P.-Y., & Kaplan, F. (2007). What is intrinsic motivation? A typology of computational approaches. *Frontiers in Neurorobotics*, 1, 6.

[27] Singh, S., Barto, A., & Chentanez, N. (2005). Intrinsically motivated reinforcement learning. *Advances in Neural Information Processing Systems*, 17.

[28] Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-predictive next feature learning. *International Conference on Machine Learning*.

[29] von Neumann, J. (1966). *Theory of Self-Reproducing Automata*. University of Illinois Press.

[30] Maes, P. (1988). Computational reflection. *The Knowledge Engineering Review*, 3(1), 1–19.

[31] Kiczales, G., des Rivières, J., & Bobrow, D. G. (1991). *The Art of the Metaobject Protocol*. MIT Press.

[32] Wang, G., et al. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

[33] Erol, K., Hendler, J., & Nau, D. S. (1994). HTN planning: Complexity and expressivity. *AAAI Conference on Artificial Intelligence*.

[34] McDermott, D., et al. (1998). PDDL — The planning domain definition language. *Technical Report CVC TR-98-003*, Yale University.

[35] Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

[36] O'Neill, M., & Ryan, C. (2001). Grammatical evolution. *IEEE Transactions on Evolutionary Computation*, 5(4), 349–358.

[37] Zoph, B., & Le, Q. V. (2017). Neural architecture search with reinforcement learning. *International Conference on Learning Representations*.

[38] Lin, X. V., Wang, C., Zettlemoyer, L., & Ernst, M. D. (2018). NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. *Language Resources and Evaluation Conference*.

[39] Friston, K., et al. (2017). Active inference: A process theory. *Neural Computation*, 29(1), 1–49.

[40] Friston, K. J. (2007). *Statistical Parametric Mapping: The Analysis of Functional Brain Images*. Academic Press.

[41] Heins, C., et al. (2022). pymdp: A Python library for active inference in discrete state spaces. *Journal of Open Source Software*, 7(73), 4098.

[42] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

[43] Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), 209–212.